

第十一章：NPC 社交系统

引言

本章导读

本章是 AI 小镇项目的**核心功能层**，是一次在教学操作系统领域的**开创性探索**。我们**首次在 uCore 教学操作系统中实现了具备自主社交能力的多线程 NPC 系统**，将操作系统的线程同步、进程间通信与 AI 决策系统深度融合。

本章的创新贡献：

- 提出了"动物脑结构"类比的多线程 AI 架构：**将 NPC 的智能分为"小脑"（主线程-基础生命维持）和"大脑"（工作线程-高级认知），实现了灵活可配置的 AI 智力等级。
- 设计了"五线程协作模型"：**
 - 0号线程（小脑）：主循环、生命周期管理
 - 1号线程（感知）：从外界接收信息
 - 2号线程（思考）：AI 决策、自我认知（"我思故我在"）
 - 3号线程（沟通）：向外界发送信息
 - 4号线程（记忆）：管理三级记忆系统
- 实现了"三级记忆系统"：**人设（永久L1）→ AI记忆（持久L2）→ 会话历史（临时L3），为 AI 提供了类人的记忆层次结构。
- 创新了"主动发起机制"：**通过随机数驱动，NPC 不只是被动响应消息，还会主动发起社交，更接近真实社交行为。
- 设计了"生产者-消费者"模式的线程间通信：**inbox/outbox/pending_memory 三对缓冲区，实现了感知→思考→沟通→记忆的完整数据流。

在上一章中，我们实现了进化调度系统，NPC 的优先级会随时间衰减，最终被 OOM Killer 淘汰。但那时的 NPC 只是"默默等死"——没有任何自救手段。本章将赋予 NPC 社交能力：通过与其他 NPC 交流获得 IPC 奖励，从而延长生存时间。

从"沉默的个体"到"社交的群体"

一个只会孤独存在的 NPC 是没有意义的。真正的 AI 小镇需要：

- NPC 之间能够交流信息
- 社交行为带来生存优势（IPC 奖励）
- 每个 NPC 有独特的"人格"和"记忆"
- AI 驱动的自主决策

这就是本章要实现的目标。

本章目标

本章将在 ch10 进化调度的基础上，实现完整的 NPC 社交系统：

组件	功能
五线程架构	每个 NPC 运行感知、思考、沟通、记忆四个工作线程
管道通信	NPC 间通过管道异步传递消息
AI 驱动决策	thinking 线程调用外部 AI API
三级记忆	人设→AI记忆→会话历史的层次结构
IPC 奖励	社交行为触发优先级奖励

实践体验

获取本章代码：

```
源码已上传大赛官网
```

在 qemu 模拟器上运行本章代码：

```
$ make BASE=1 CHAPTER=11
$ make run
>> ch11_world

=====
ch11: NPC world Started
ch11: Creating pipes between NPCs...
ch11: Spawning 3 NPCs...
=====

[NPC 1] Born! Starting 4 threads...
[NPC 1][thinking] AI response: "[to npc2]: 你好！"
[NPC 1][comm] Sending "你好！" to NPC 2
[IPC Reward] NPC 1 +1 (prio: 50->51)

[NPC 2][perception] Received "你好！" from NPC 1
[IPC Reward] NPC 2 +1 (prio: 50->51)
[NPC 2][thinking] AI response: "[to npc1]: 你也好！"
...
```

看到 NPC 之间互相交流，并通过社交获得 IPC 奖励，说明社交系统工作正常！

本章代码树

```
os-btbu/
├─ os/
│   ├── npc_memory.c      # NPC记忆管理（L2存储）
│   └─ npc_memory.h      # 记忆系统头文件
```

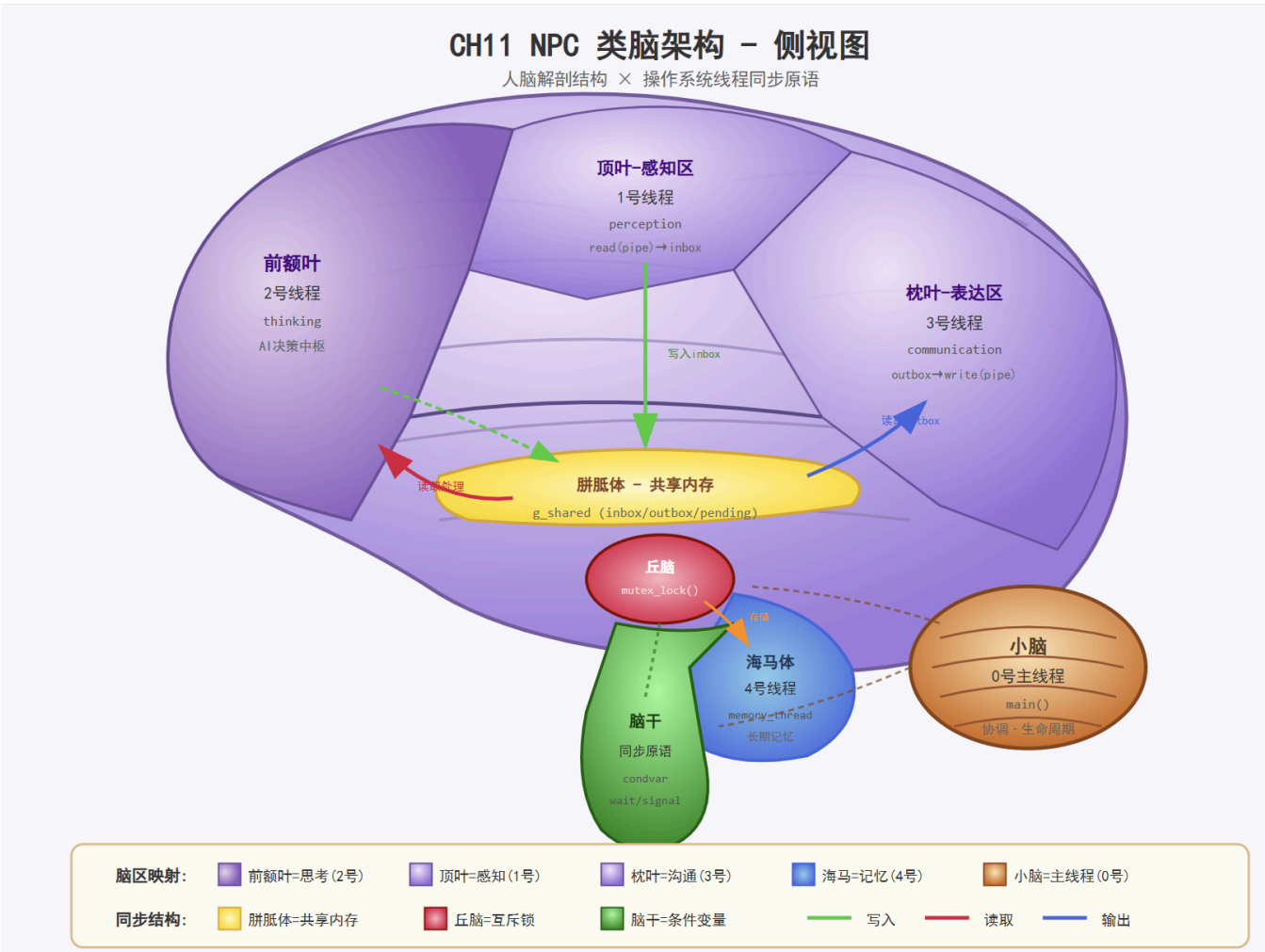
```
|   |— ai_sched.c           # 进化调度（添加IPC奖励触发）
|   |— syscall.c          # 系统调用（新增记忆操作）
|   └─ user/
|       |— src/
|           |— ch11_world.c # 世界管理器（创建管道）
|           |— ch11_npc.c  # NPC进程（四线程）
|       |— lib/
|           |— npc.c       # NPC用户态库
|       └─ include/
|           └─ npc.h       # NPC头文件
```

设计理念：动物脑结构类比

小脑与大脑的分工

本系统采用**动物脑结构**来设计 AI 线程架构：

脑区	对应线程	功能特点	类比
小脑	0号线程（主线程）	基础功能、生命维持	吃喝睡觉、心跳呼吸
大脑	1-4号工作线程	高级认知、复杂思考	感知、思考、沟通、记忆



这种设计允许**灵活定制 AI 的智力等级**：

- **低智能 NPC**（如猫、狗）：只保留小脑功能，简单的吃喝睡觉
- **中智能 NPC**：小脑 + 部分大脑功能（如感知+简单反应）
- **高智能 NPC**：完整的五线程架构，具备自我认知能力

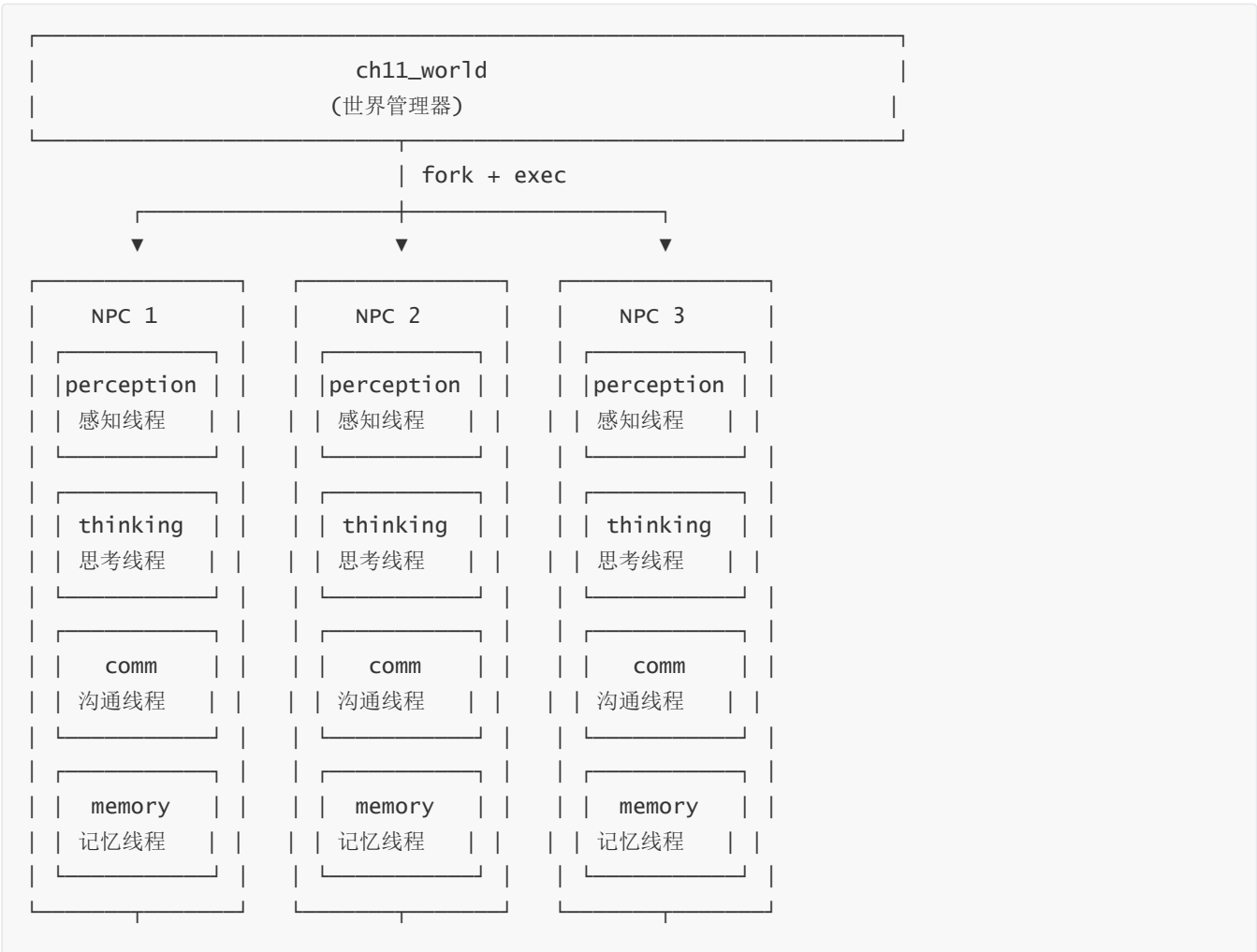
五线程架构定义

编号	线程名称	功能职责	代码入口
0号线程	小脑线程	主循环、生命周期管理、基础调度	<code>main()</code>
1号线程	感知线程	从外界接收信息（管道读取）	<code>perception_thread()</code>
2号线程	思考线程	AI决策、自我认知（"我思故我在"）	<code>thinking_thread()</code>
3号线程	沟通线程	向外界发送信息（管道写入）	<code>communication_thread()</code>
4号线程	记忆线程	管理三级记忆系统、持久化存储	<code>memory_thread()</code>

关于自我认知：当思考线程的 AI 模型中没有"自我符号"时，该 NPC 不具备自我认知能力，只是简单的刺激-反应机器。

系统架构

整体结构



线程间通信机制

NPC 内部的线程通信采用**共享内存 + 同步原语**。

操作系统同步原语详解

本系统使用三类操作系统原语实现线程协作：

原语	系统调用	用途
互斥锁 (Mutex)	mutex_blocking_create()	保护共享内存的互斥访问
	mutex_lock(id)	进入临界区前加锁
	mutex_unlock(id)	离开临界区后解锁
条件变量 (Condvar)	condvar_create()	创建等待/通知机制
	condvar_signal(id)	通知等待线程
	condvar_wait(id, mutex)	等待条件满足
管道 (Pipe)	sys_pipe(fds)	创建进程间通信管道
	read(fd, buf, n)	从管道读取
	write(fd, buf, n)	向管道写入

临界区保护模式

所有对共享数据的访问都遵循以下模式：

```
/* ch11: 标准临界区访问模式 */
mutex_lock(shared->mutex_id);          /* 1. 加锁 */

/* 2. 访问共享数据（临界区） */
if (shared->has_new_msg) {
    /* 读取 inbox */
    shared->has_new_msg = 0;
}

mutex_unlock(shared->mutex_id);         /* 3. 解锁 */
```

生产者-消费者模式

系统中有三对生产者-消费者关系:

共享缓冲区	生产者	消费者	状态标志
inbox	1号感知线程	2号思考线程	has_new_msg

共享缓冲区	生产者	消费者	状态标志
outbox	2号思考线程	3号沟通线程	has_pending_send
pending_mem	2号思考线程	4号记忆线程	has_pending_memory

共享数据结构

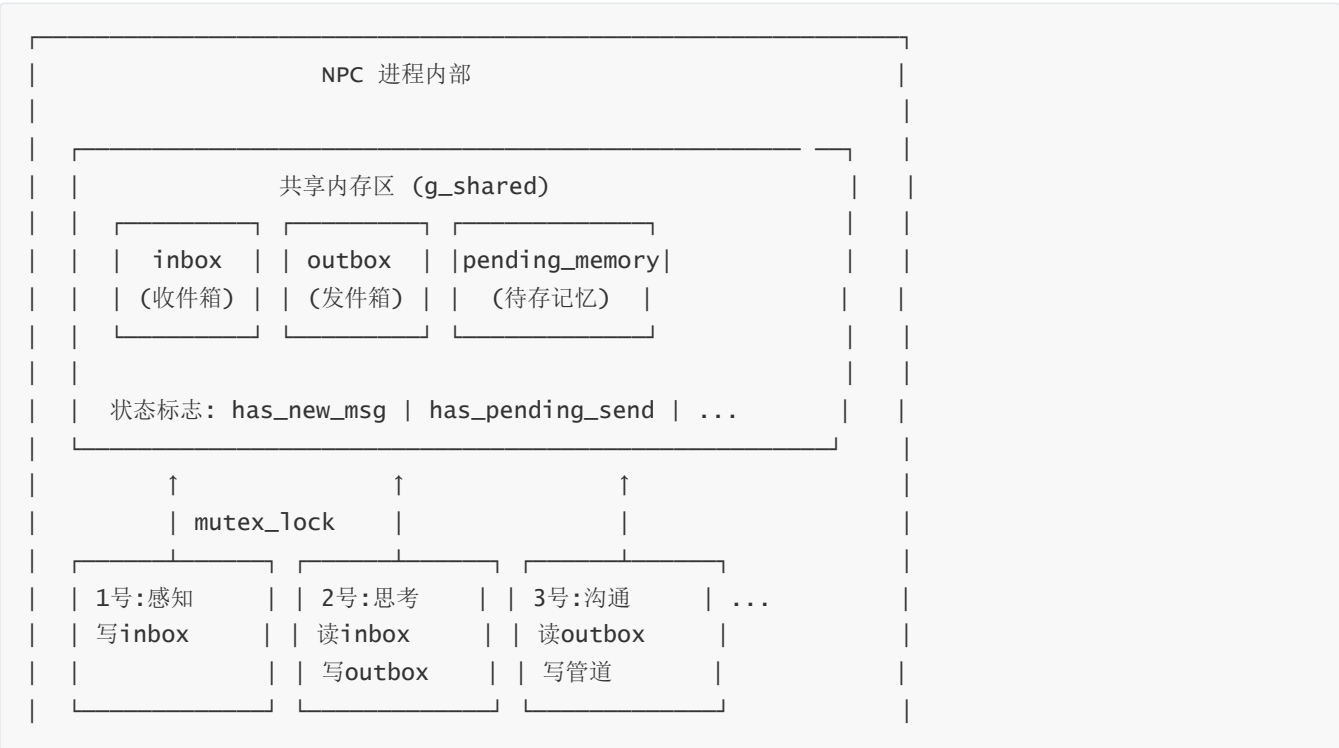
```
/* user/src/ch11_npc.c */
struct npc_shared {
    /* ch11: 互斥锁 */
    int mutex_id;
    int cond_id;

    /* ch11: 感知缓冲区 */
    char inbox[512];          /* 收到的消息 */
    int inbox_len;
    int inbox_from;          /* 发送者NPC ID */

    /* ch11: 待发消息队列 */
    char outbox[512];         /* 待发送的消息 */
    int outbox_len;
    int outbox_to;           /* 目标NPC ID */

    /* ch11: 状态标志 */
    int has_new_msg;          /* 有新消息待处理 */
    int has_pending_send;    /* 有消息待发送 */
    int running;             /* NPC是否运行中 */
};
```

共享内存区结构



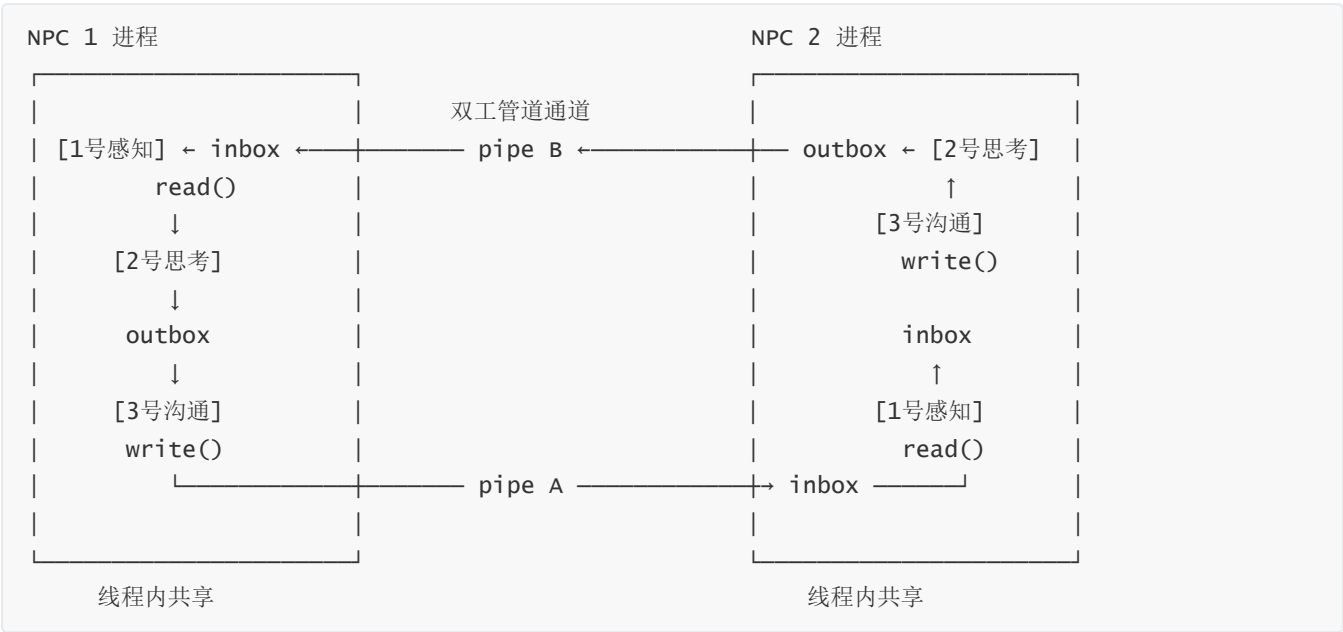
NPC 间通信

线程级共享 vs 进程间通信

需要区分两个不同层级的数据交互：

层级	通信方式	参与者	示例
线程级共享	共享内存 + 互斥锁	同一 NPC 的 0-4 号线程	inbox/outbox 缓冲区
进程间通信	管道 (pipe)	不同 NPC 进程之间	NPC1 发消息给 NPC2

全双工管道通信



特点：

- 异步通信：发送后不等待回复（像微信）
- perception 线程非阻塞轮询管道
- 成功发送/接收触发 IPC 奖励

管道管理

```
/* ch11: 世界管理器创建 NPC 间管道 */
int pipes[NPC_COUNT][NPC_COUNT][2]; /* pipes[from][to][read/write] */

/* ch11: NPC 1 发消息给 NPC 2 */
write(pipes[1][2][1], msg, len); /* 写入 pipe[1->2] 的写端 */

/* ch11: NPC 2 接收来自 NPC 1 的消息 */
read(pipes[1][2][0], buf, size); /* 读取 pipe[1->2] 的读端 */
```

三级记忆系统

这是本章的一个重要创新——为 AI 提供类人的记忆层次结构。

级别	名称	内容示例	存储位置	生命周期
L1	人设	"我是小明，性格开朗，喜欢交朋友"	代码硬编码	永久
L2	AI记忆	"npc2很友好，npc3有点冷淡"	内核专用内存	持久
L3	会话历史	本轮所有对话记录	进程内存	临时

内核记忆区

```
/* os/npc_memory.c */
#define NPC_MAX 16
#define MEMORY_SIZE 4096

struct npc_memory {
    int npc_id;
    char l2_memory[MEMORY_SIZE]; /* 二级记忆 */
    int l2_len;
};

static struct npc_memory memories[NPC_MAX];
```

记忆系统调用

```
/* ch11: 保存二级记忆 */
int npc_memory_save(int npc_id, char *content, int len);

/* ch11: 读取二级记忆 */
int npc_memory_load(int npc_id, char *buf, int maxlen);
```

AI 驱动决策

主动发起机制

本章的一个创新是"主动发起"——NPC 不只是被动响应消息，还会主动找人聊天：

```
/* ch11: thinking 线程的触发条件 */
int should_think = 0;

/* 条件1: 收到新消息 */
if (shared->has_new_msg)
    should_think = 1;

/* 条件2: 随机主动发起 (概率约20%) */
if (rand() % 100 < 20)
    should_think = 1;
```



```
if (should_think) {
    /* 调用 AI API */
    /* AI 可能返回 [to none] 表示不说话 */
}
```

为什么用轮询而非阻塞等待？

- 思考线程有两个触发条件：被动(收到消息) + 主动(随机发起)
- 如果用 `condvar_wait()` 阻塞，无法实现"主动发起聊天"
- 轮询模式下，每轮都能检查这两个条件

AI 返回格式

[to npc2]: 你好，今天天气不错！
[memory]: npc2主动跟我打招呼，她好像挺友好的

解析规则：

- [to npcX]: 后面是发给 npcX 的消息
- [to none]: 表示这轮不发消息给任何人（沉默/思考中）
- [memory]: 后面是需要存入二级记忆的内容
- 可以有多个 [to] 和 [memory] 标签

支持群聊的消息格式

AI返回示例：
[to npc2]: 大家好！
[to npc3]: 大家好！
[memory]: 发起了一次群聊

完整流程

一轮循环

1. perception线程

- └─ 非阻塞读取管道
- └─ 如果有消息：存入 `inbox`，设置 `has_new_msg = 1`
- └─ 通知 `thinking` 线程 (`condvar_signal`)

2. thinking线程

- └─ 检查条件（有新消息或随机触发）
- └─ 构建 `prompt`:
 - └─ L1: 人设（硬编码）
 - └─ L2: AI记忆 (`npc_memory_load`)
 - └─ L3: 会话历史（进程内存）
- └─ 调用 `ai_chat(prompt)`

- └─ 解析 AI 返回:
- | └─ [to npcX]: 存入 outbox, 设置 outbox_to = x
- | └─ [memory]: 存入待保存队列
- └─ 通知 comm 和 memory 线程

3. communication线程

- └─ 等待 has_pending_send
- └─ 写入目标 NPC 管道: write(pipe[me][to], outbox, len)
- └─ 触发 IPC 奖励: npc_ipc_notify()
- └─ 清空 outbox

4. memory线程

- └─ 等待有新记忆需要保存
- └─ 调用 npc_memory_save() 存入 L2
- └─ 更新 L3 会话历史

5. 主线程

- └─ npc_yield() 触发进化调度
- └─ 检查优先级, 决定是否继续
- └─ 循环

IPC 奖励机制

这是连接 ch10 进化调度和 ch11 社交系统的关键:

```
/* os/ai_sched.c */
void ai_sched_ipc_reward(int from_npc, int to_npc, int bytes) {
    if (bytes >= NPC_IPC_REWARD_BYTES) {
        int reward = bytes / NPC_IPC_REWARD_BYTES;

        /* ch11: 发送方奖励 */
        struct proc *sender = find_npc_proc(from_npc);
        if (sender) {
            sender->dynamic_prio += reward;
            if (sender->dynamic_prio > NPC_PRIO_MAX)
                sender->dynamic_prio = NPC_PRIO_MAX;
        }

        /* ch11: 接收方奖励 */
        struct proc *receiver = find_npc_proc(to_npc);
        if (receiver) {
            receiver->dynamic_prio += reward;
            if (receiver->dynamic_prio > NPC_PRIO_MAX)
                receiver->dynamic_prio = NPC_PRIO_MAX;
        }

        printf("[IPC Reward] NPC %d +%d, NPC %d +%d\n",
            from_npc, reward, to_npc, reward);
    }
}
```

核心理念：社交带来生存优势。积极参与交流的 NPC 能获得优先级奖励，延长生存时间；而"孤僻"的 NPC 则会因优先级耗尽被淘汰。

系统调用接口

本章新增三个系统调用：

系统调用	调用号	参数	功能
npc_memory_save	486	(npc_id, content, len)	保存L2记忆
npc_memory_load	487	(npc_id, buf, maxlen)	读取L2记忆
npc_ipc_notify	488	(from_id, to_id, bytes)	通知内核IPC发生

与 ch10 的关系

ch10	ch11
优先级衰减机制	继续使用
OOM Killer	继续使用
npc_register/get_status/yield	继续使用
IPC奖励接口（预留）	正式启用
单线程NPC	升级为五线程

知识点总结

- 1. 多线程协作：
 - 互斥锁保护共享数据
 - 条件变量实现线程间通知
 - 生产者-消费者模式
- 2. 进程间通信：
 - 管道实现异步消息传递
 - 全双工通信（每对 NPC 两条管道）
 - 非阻塞读取
- 3. AI 集成：
 - 结构化 prompt 构建
 - AI 返回解析
 - 三级记忆管理
- 4. 自然选择模拟：
 - 社交行为 → IPC 奖励 → 延长生存
 - 孤僻行为 → 无奖励 → 被淘汰

思考题

1. 当前实现中，NPC 之间的管道是在 world 进程中创建的。如果要支持动态加入新 NPC，需要如何修改设计？
2. 三级记忆系统中，L2 记忆存储在内核中。如果 NPC 数量很大，如何优化内存使用？
3. 如何设计一个“负向社交”机制——某些类型的交流会导致优先级下降？
4. 当前的随机主动发起机制使用固定的 20% 概率。如何让 AI 自己决定是否主动发起聊天？

预期运行输出

```
=====
ch11: NPC world Started
ch11: Creating pipes between NPCs...
ch11: Spawning 3 NPCs...
=====

[NPC 1] Born! Starting 4 threads...
[NPC 1][perception] Thread started
[NPC 1][thinking] Thread started
[NPC 1][comm] Thread started
[NPC 1][memory] Thread started

[NPC 2] Born! Starting 4 threads...
[NPC 3] Born! Starting 4 threads...

[NPC 1][thinking] Building prompt with L1+L2+L3...
[NPC 1][thinking] Calling AI API...
[NPC 1][thinking] AI response: "[to npc2]: 你好! [memory]: 第一次主动打招呼"
[NPC 1][comm] Sending "你好! " to NPC 2
[IPC Reward] NPC 1 +1 (prio: 50->51)
[NPC 1][memory] Saved to L2: "第一次主动打招呼"

[NPC 2][perception] Received "你好! " from NPC 1
[IPC Reward] NPC 2 +1 (prio: 50->51)
[NPC 2][thinking] Building prompt: "NPC 1 said: 你好! "
[NPC 2][thinking] Calling AI API...
[NPC 2][thinking] AI response: "[to npc1]: 你也好! [memory]: npc1很友好"
[NPC 2][comm] Sending "你也好! " to NPC 1
[IPC Reward] NPC 2 +1 (prio: 51->52)

... (NPC们互相聊天, 通过IPC奖励维持优先级) ...

[NPC 3] tick=30, prio=20, alive=3
[NPC 3][thinking] AI API timeout...
[NPC 3] No social activity, priority decaying...

... (不社交的NPC优先级下降) ...

[OOM Killer] NPC 3 killed (prio=0)
[NPC 3] I'm dying...

... (剩余NPC继续社交) ...
```

```
=====
ch11: All NPCs dead, world ends
=====
```

下一步展望

本章实现了 NPC 社交系统的基础框架。未来可以考虑：

- **情感系统**：好感度、信任度
- **更复杂的社交网络**：小团体、派系
- **NPC 组队/对抗机制**：合作任务、竞争资源
- **长期记忆持久化**：将 L2 记忆写入文件系统